

Flutter之万物皆Widget（一种你没见过的方式来深入Widget）



i校长



关注



1

2020.09.23 17:31:58 字数 2,459 阅读 825



i校长



总资产332

关注

程序员也适用的人生商业模式

阅读 132

如何阅读框架源码

阅读 176

背景

为什么说Flutter万物皆Widget？首先你要知道，Flutter是什么，它是一个现代的响应式框架、一个2D渲染引擎、现成的widget和开发工具，基于Skia，一个性能彪悍的2D图像绘制引擎，2005年被Google收购，被广泛应用于Chrome和Android之上，等等吧，说白了，Flutter就是一个UI框架，所以说，万物皆Widget，而Widget的中文意思是小部件，它为什么不能像Android或者ios一样叫做View呢？因为widget既可以是一个结构元素（如按钮或菜单）、也可以是一个文本样式元素（如字体或颜色方案）、布局的一个方面（如填充）等等，我们可以统筹它们为widget，而不是view，根据基本的命名规范，这就是一种合理的命名抽象。那么接下来我们学什么？

- Widget是什么
- Widget类结构
- 跟着我实现一个widget（直接继承widget抽象类）
- Element类结构
- 深入理解Element

Widget是什么

其实上面说了，一切皆Widget，那我们可不可以认为，在flutter的框架中，用到的东西都是Widget呢，当然不是哈，由于它是基于Dart，所以有很多Dart的库，还是可以使用的，比如AES，RSA加密解密，Json序列化等等，但你可以这么说，一切构建图形相关的东西都是Widget，这就是Widget

Widget类结构

为什么说下类结构呢？类结构可以很清晰帮助我们梳理逻辑，从全局的角度看待整个结构

简书

首页

下载APP

会员

IT技术

搜索



Aa



beta

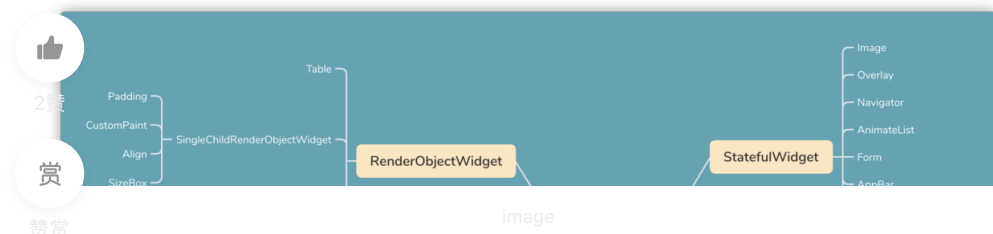
登录

注册

热门故事

前世渣男把我迷晕后送到新科状元床上！

你看过最恐怖的悬疑小说是哪部？



RenderObjectWidget 看名字我们判断，它是持有RenderObject对象的Widget，而通过其他通道了解到，RenderObject实际上是完成界面的布局、测量与绘制，像Padding，Table，Align都是它的子类

- StatefulWidget 多了一个State状态的Widget，子类都是可以动态改变的如CheckBox，Switch
- StatelessWidget 就是一个普通的Widget,不可变如Icon，Text。
- ProxyWidget InheritedWidget就是它的子类，我们暂且认为它是子类能从父类拿数据的关键，以后再研究，大多数的主题都是继承自ProxyWidget

跟我一起实现一个Widget

我不想和别人的教程思路一样，既然万物皆Widget，那我们就从实现一个Widget开始，然后一步步深入，看到什么就去了解什么？来上代码

```
1 class TestWidget extends Widget{
2   @override
3   Element createElement() {
4     // TODO: implement createElement
5     throw UnimplementedError();
6   }
7 }
```

创建一个TestWidget然后继承Widget，然后会让你重写函数createElement，返回一个Element，通过这个我们看的出，其实我们创建的Widget，最终肯定是创建了一个Element，那Element到底是什么呢？同样的思路，我们继承Element看一下

```
1 class TestElement extends Element{
2
3   TestElement(Widget widget) : super(widget);
4
5   @override
6   bool get debugDoingBuild => throw UnimplementedError();
7
8   @override
9   void performRebuild() {
10  }
11
12 }
```

写下你的评论...

评论 1

赞 2

...

数，都是干嘛的呢？

```
1 abstract class Element extends DiagnosticableTree implements BuildContext
2
3 abstract class BuildContext {
4
5   /// Whether the [widget] is currently updating the widget or render tree.
6   ///
7   /// For [StatefulWidget]s and [StatelessWidget]s this flag is true while
```

妖艳女主播专挑凶宅直播跳舞结果.....

我捡回来的奶狗弟弟变身狼狗！遭不住

穿成王宝钏：做大唐第一渣女

推荐阅读

flutter面试题

阅读 607

Flutter 实现扫码

阅读 348

iOS 解决viewController使用xib时控件为nil问题

阅读 163

Android Framework 之 修改调试小技巧

阅读 309

阅读源码分析使用 restframework-bulk 包进行`批量`操作中的`批量更

阅读 139

```

8  |  /// their respective build methods are executing.
9  |  /// [RenderObjectWidget]s set this to true while creating or configuring their
10 |  /// associated [RenderObject]s.
11 |  /// Other [Widget] types may set this to true for conceptually similar phases
12 |  /// of their lifecycle.
13 |  ///
14 |  /// When this is true, it is safe for [widget] to establish a dependency to an
15 |  /// [InheritedWidget] by calling [dependOnInheritedElement] or
16 |  /// [dependOnInheritedWidgetOfExactType].
17 |  ///
18 |  /// Accessing this flag in release mode is not valid.
19 |  bool get debugDoingBuild;
20 |

```

经过代码的跟踪我们发现一些注解：

- Element继承自DiagnosticableTree，并实现BuildContext
- DiagnosticableTree是个“诊断树”，主要作用是提供调试信息。
- BuildContext类似原生系统的上下文，它定义了debugDoingBuild，通过注解我们知道，它应该就是一个debug用的一个标志位。
- performRebuild 经过源码查看后发现，由rebuild()调用如下

```

1  |  void rebuild() {
2  |      if (!_active || !_dirty)
3  |          return;
4  |      performRebuild();
5  |  }
6  |
7  |  @override
8  |  void update(ProxyWidget newWidget) {
9  |      rebuild();
10 |  }
11 |

```

首先说明下，这个并不是Element的源码，我摘自StatelessElement，是Element的子类，这说明在update函数后，Element就会直接执行performRebuild函数，那我们完善下自定义的Element逻辑

```

1  |  class TestElement extends Element {
2  |
3  |      TestElement(Widget widget) : super(widget);
4  |
5  |      @override
6  |      bool get debugDoingBuild => throw UnimplementedError();
7  |
8  |      @override
9  |      void performRebuild() {
10 |      }
11 |
12 |      @override
13 |      void update(Widget newWidget) {
14 |          super.update(newWidget);
15 |          print("TestWidget update");
16 |          performRebuild();
17 |      }
18 |
19 |      @override
20 |      TestWidget get widget => super.widget as TestWidget;
21 |
22 |      Widget build() => widget.build(this);
23 |  }

```

在update的时候执行performRebuild(),但是performRebuild执行什么呢？我们结合一下StatelessElement的实现，发现，它调用了传递进来的Widget参数build函数，那么我们就在

TestWidget中添加函数，并完善下逻辑后是这样的

```
1 class TestWidget extends Widget {
2
3   @override
4   Element createElement() {
5     /// 将自己传递进去，让Element调用下面的build函数
6     return TestElement(this);
7   }
8   /// 这个context其实就是Element
9   Widget build(BuildContext context) {
10    print("TestWidget build");
11    return Text("TestWidget");
12  }
13 }
14
15 class TestElement extends Element {
16
17   Element _child;
18
19   TestElement(Widget widget) : super(widget);
20
21   @override
22   bool get debugDoingBuild => throw UnimplementedError();
23
24   @override
25   void performRebuild() {
26     ///调用build函数
27     var _build = build();
28     ///更新子视图
29     _child = updateChild(_child, _build, slot);
30   }
31
32   @override
33   void update(Widget newWidget) {
34     super.update(newWidget);
35     print("TestWidget update");
36     ///更新
37     performRebuild();
38   }
39
40   ///将widget强转成TestWidget
41   @override
42   TestWidget get widget => super.widget as TestWidget;
43   /// 调用TestWidget的build函数
44   Widget build() => widget.build(this);
45 }
```

然后将其放入main.dart中如图

```
// then having to individually change instances of widgets.  
return Scaffold(  
  appBar: AppBar(  
    // Here we take the value from the MyHomePage object that was created by  
    // the App.build method, and use it to set our appBar title.  
    title: Text(widget.title),  
  ), // AppBar  
  body: Center(  
    child: Text(widget.title),  
  ),  
);
```

image

最终效果展示，如图

[图片上传失败...(image-72eee6-1600853501724)]

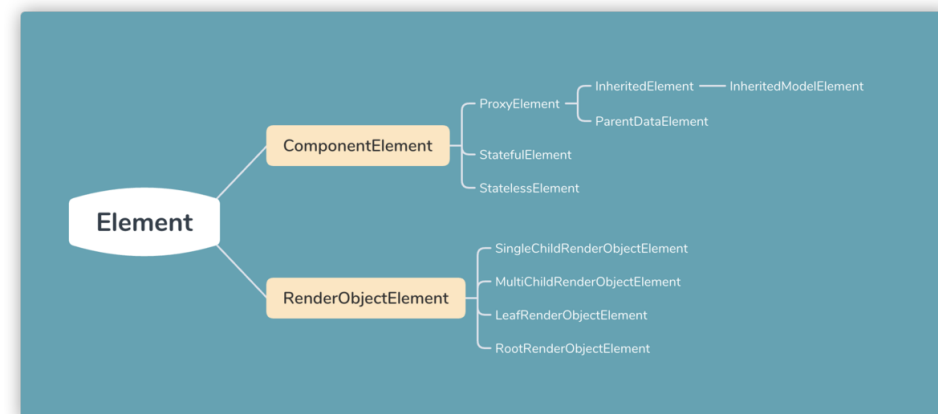
展示出来了，我们简单总结一下，到目前你学到了什么？

- Widget会创建Element对象（调用createElement并不是Widget，而是Framework）
- Widget并没有实际的操控UI
- Element是在update的时候重新调用Widget的build函数来构建子Widget
- updateChild会根据传入的Widget生成新的Element
- Widget的函数build，传入的context其实就是它创建的Element对象，那么为什么这么设计呢？一方面它可以隔离掉一些Element的细节，避免Widget频繁调用或者误操作带来的不确定问题，一方面context上下文可以存储树的结构，来从树种查找元素。

其实可以很简单的理解为，Widget就是Element的配置信息，在Dart虚拟机中会频繁的创建和销毁，由于量比较大，所以抽象一层Element来读取配置信息，做一层过滤，最终再真实的绘制出来，这样做的好处就是避免不必要的刷新。接下来我们深入了解下Element

Element类结构

在深入了解Element之前我们也从全局看下它的结构



image

可以看到，Element最主要的两个抽象：

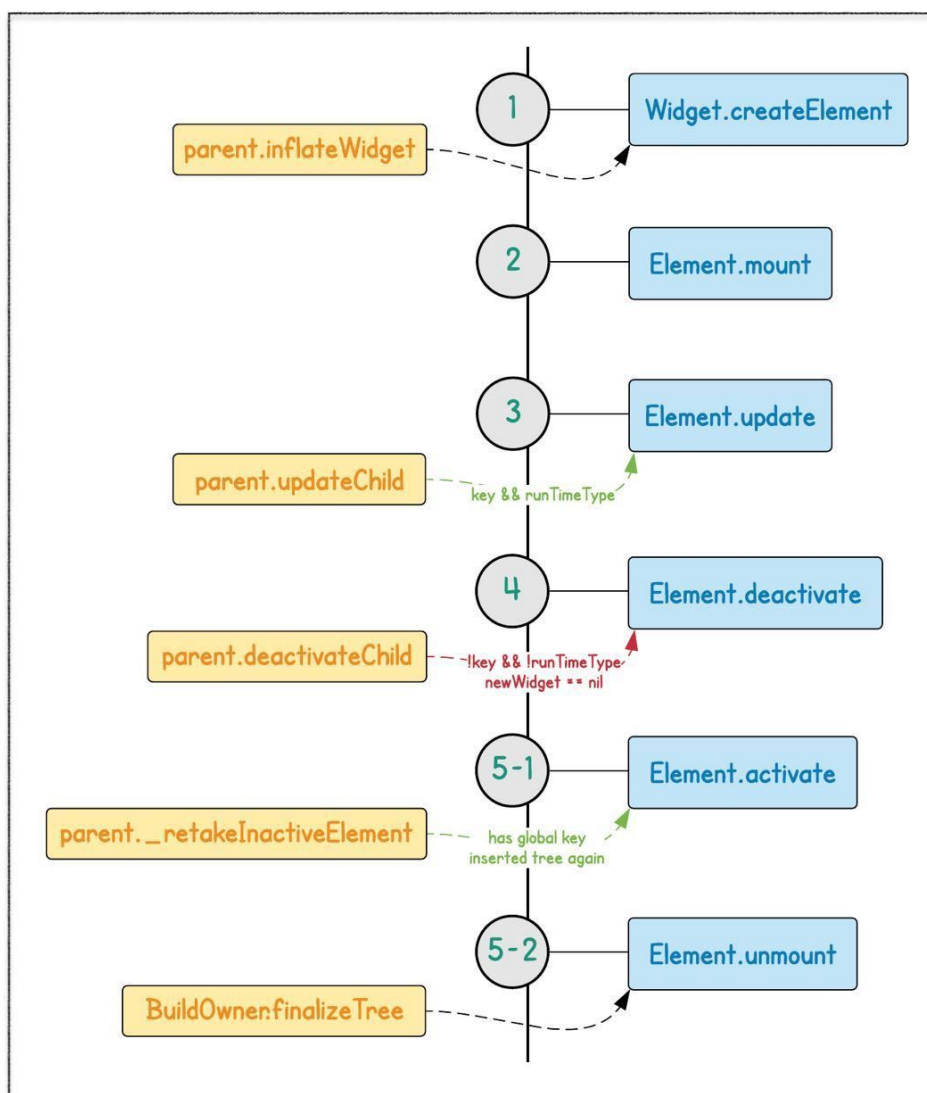
- ComponentElement
- RenderObjectElement

都是干嘛的呢？经过看源码，发现ComponentElement，其实做了一件事情就是在mount函数中，判断Element是第一次创建，然后调用_firstBuild,最终通过rebuild调用performRebuild，通过上面我们也知道performRebuild最终调用updateChild来绘制UI

而RenderObjectElement就比较复杂一点，它创建了RenderObject，通过RenderObjectWidget的createRenderObject方法，通过以前的学习，我们也知道RenderObject其实是真正绘制UI的对象，所以我们暂且认为RenderObjectElement其实是可以直接操控RenderObject，一种更直接的方式来控制UI。

深入理解Element

为什么要深入理解Element呢，由于大多数情况下，我们开发者并不会直接操作Element，但对于想要全局了解FlutterUI框架至关重要，特别实在一些状态管理的框架中，如Provider，他们都定制了自己的Element实现，那么这么重要，我们需要从哪方面了解呢？一个很重要的知识点就是生命周期，只有了解了正确的生命周期，你才能在合适的时间做合适的操作



image

为了验证该图，我们加入日志打印下，代码如下：

```
1 /// 创建LifecycleElement 实现生命周期函数
2 class LifecycleElement extends TestElement{
```

```
3
4   LifecycleElement(Widget widget) : super(widget);
5
6   @override
7   void mount(Element parent, newSlot) {
8     print("LifecycleElement mount");
9     super.mount(parent, newSlot);
10  }
11
12  @override
13  void unmount() {
14    print("LifecycleElement unmount");
15    super.unmount();
16  }
17
18  @override
19  void activate() {
20    print("LifecycleElement activate");
21    super.activate();
22  }
23
24  @override
25  void rebuild() {
26    print("LifecycleElement rebuild");
27    super.rebuild();
28  }
29
30  @override
31  void deactivate() {
32    print("LifecycleElement deactivate");
33    super.deactivate();
34  }
35
36  @override
37  void didChangeDependencies() {
38    print("LifecycleElement didChangeDependencies");
39    super.didChangeDependencies();
40  }
41
42  @override
43  void update(Widget newWidget) {
44    print("LifecycleElement update");
45    super.update(newWidget);
46  }
47
48  @override
49  Element updateChild(Element child, Widget newWidget, newSlot) {
50    print("LifecycleElement updateChild");
51    return super.updateChild(child, newWidget, newSlot);
52  }
53
54  @override
55  void deactivateChild(Element child) {
56    print("LifecycleElement deactivateChild");
57    super.deactivateChild(child);
58  }
59
60 }
61
62 class TestWidget extends Widget {
63
64   @override
65   Element createElement() {
66     /// 将自己传递进去，让Element调用下面的build函数
67     /// 更新TestElement为LifecycleElement
68     return LifecycleElement(this);
69   }
70   /// 这个context其实就是Element
71   Widget build(BuildContext context) {
72     return Text("TestWidget");
73   }
74 }
75
```

然后改造下main.dart, 如下

```
1  ///添加变量
2  bool isShow = true;
3  /// 加入变量控制
4  isShow ? TestWidget() : Container(),
5  /// 将FloatingActionButton改为这样的实现
6  onPressed: () {
7      setState(() {
8          isShow = !isShow;
9      });
10 }
```

运行一下项目查看日志

```
I/flutter ( 6614): LifecycleElement mount
I/flutter ( 6614): LifecycleElement update
I/flutter ( 6614): LifecycleElement updateChild
```

image

- 调用 element.mount(parentElement,newSlot)
- 调用 update(Widget newWidget)
- 调用 updateChild(Element child, Widget newWidget, newSlot)

然后我们点击下按钮

```
I/flutter ( 6614): LifecycleElement deactivate
I/flutter ( 6614): LifecycleElement unmount
```

image

- 调用 deactivate()
- 调用 unmount()

我们再点击下按钮

```
I/flutter ( 6614): LifecycleElement mount
```

image

这次只有mount，为什么？由于Widget本身不可变，我判断是因为这个导致的，那如何判断呢？下面介绍一个小技巧，其实flutter的framework层是可以加入调试代码的，我们加入日志看下，如下：


```

1  /// widget 基类其实有一个canUpdate函数，我们猜测肯定是这里导致的，加入日志如下
2  static bool canUpdate(Widget oldWidget, Widget newWidget) {
3
4      if(oldWidget.toString()=="TestWidget") {
5          print("canUpdate${oldWidget.runtimeType == newWidget.runtimeType
6              && oldWidget.key == newWidget.key}");
7      }
8
9      return oldWidget.runtimeType == newWidget.runtimeType
10         && oldWidget.key == newWidget.key;
11 }

```

是个静态函数，肯定是在Element中被调用的，我们找下

```

1  @mustCallSuper
2  void update(covariant Widget newWidget) {
3
4      if (newWidget.toString() == "TestWidget") {
5          print("TestWidget update start");
6      }
7
8      assert(_debugLifecycleState == _ElementLifecycle.active
9          && widget != null
10         && newWidget != null
11         && newWidget != widget
12         && depth != null
13         && _active
14         && Widget.canUpdate(widget, newWidget));
15
16      assert(() {
17          _debugForgottenChildrenWithGlobalKey.forEach(_debugRemoveGlobalKeyReservation);
18          _debugForgottenChildrenWithGlobalKey.clear();
19          return true;
20      }());
21      if (newWidget.toString() == "TestWidget") {
22          print("TestWidget:${newWidget.hashCode}");
23      }
24      _widget = newWidget;
25  }

```

如上代码是Element的源码，这里调用了canUpdate函数，如果不需要更新的话，就直接中断了执行，我们重新运行下demo,并在加一个print来验证一下newWidget是什么样子的，这里加入newWidget.toString() == "TestWidget"，主要是为了过滤垃圾日志，重新运行项目。如图

```

Restarted application in 1,715ms.
I/flutter (25008): LifecycleElement mount
I/flutter (25008): canUpdatetrue
I/flutter (25008): canUpdatetrue
I/flutter (25008): LifecycleElement update
I/flutter (25008): TestWidget update start
I/flutter (25008): canUpdatetrue
I/flutter (25008): TestWidget:160828048
I/flutter (25008): LifecycleElement updateChild

```

image

点击后按钮

```
I/flutter (25008): canUpdatefalse
I/flutter (25008): canUpdatefalse
I/flutter (25008): LifecycleElement deactivate
I/flutter (25008): LifecycleElement unmount
```

image

再点击

```
I/flutter (25008): LifecycleElement mount
```

image

发现并没有调用canUpdate，那我们如何让它重新加载回来呢？我们查查资料，改造下例子

```
1  @override
2  void mount(Element parent, newSlot) {
3    print("LifecycleElement mount");
4    super.mount(parent, newSlot);
5    assert(_child == null);
6    print("LifecycleElement firstBuild");
7    performRebuild();
8  }
```

mount函数加入performRebuild()函数，最终会触发updateChild，加assert断言是防止后面再加载进来的时候多次触发updateChild，然后改造下main.dart

```
1  @override
2  Widget build(BuildContext context) {
3    // This method is rerun every time setState is called, for instance as done
4    // by the _incrementCounter method above.
5    //
6    // The Flutter framework has been optimized to make rerunning build methods
7    // fast, so that you can just rebuild anything that needs updating rather
8    // than having to individually change instances of widgets.
9    return Scaffold(
10     appBar: AppBar(
11       // Here we take the value from the MyHomePage object that was created by
12       // the App.build method, and use it to set our appBar title.
13       title: Text(widget.title),
14     ),
15     body: Center(
16       // Center is a layout widget. It takes a single child and positions it
17       // in the middle of the parent.
18       child: isShow ? TestWidget() : Container(),
19     ),
20     floatingActionButton: FloatingActionButton(
21       onPressed: () {
22         setState(() {
23           isShow = !isShow;
24         });
25       },
26       tooltip: 'Increment',
27       child: Icon(Icons.add),
28     ), // This trailing comma makes auto-formatting nicer for build methods.
```

```
29 | );  
30 | }
```

去掉Column，这里是由于我们没有处理widget的index逻辑，导致在Column里不正常，后续我们再研究为什么，先看下生命周期的回调

第一次运行

[图片上传失败...(image-be90d-1600853501724)]

点击按钮

```
I/flutter ( 5461): LifecycleElement mount  
I/flutter ( 5461): LifecycleElement firstBuild  
I/flutter ( 5461): LifecycleElement updateChild
```

image

又发现一个问题，为什么我们的断言没生效呢？怎么又出现了firstBuild？哈哈，这里不要纠结，由于TestWidget并非const，导致setState后，又重新被创建了，而对应的Element也同样是创建了新的值，最终导致被重新执行。其实这个TestWidget已经不是上一个了，那我们加入const修饰再看看

```
1 | /// 改成const  
2 | const TestWidget()  
3 |  
4 | /// 加入当前widget hashCode输出，用来判断两次是否一致  
5 | @override  
6 | void mount(Element parent, newSlot) {  
7 |   print("LifecycleElement widget hashCode:${widget.hashCode}");  
8 |   print("LifecycleElement hashCode:${this.hashCode}");  
9 |   print("LifecycleElement mount");  
10 |   super.mount(parent, newSlot);  
11 |   assert(_child == null);  
12 |   print("LifecycleElement firstBuild");  
13 |   performRebuild();  
14 | }  
15 |
```

最终（启动，点击按钮两次的效果）运行效果如下：

```
I/flutter ( 9606): LifecycleElement widget hashCode13505123  
I/flutter ( 9606): LifecycleElement hashCode105  
I/flutter ( 9606): LifecycleElement mount  
I/flutter ( 9606): LifecycleElement firstBuild  
I/flutter ( 9606): LifecycleElement updateChild  
I/flutter ( 9606): LifecycleElement child=Text("TestWidget", dependencies: [DefaultTextStyle, MediaQuery])  
I/flutter ( 9606): LifecycleElement deactivate  
I/flutter ( 9606): LifecycleElement unmount  
I/flutter ( 9606): LifecycleElement widget hashCode13505123  
I/flutter ( 9606): LifecycleElement hashCode205  
I/flutter ( 9606): LifecycleElement mount  
I/flutter ( 9606): LifecycleElement firstBuild  
I/flutter ( 9606): LifecycleElement updateChild
```

image

两次运行Widget保持一致，这就避免了Widget的重建

小结

经过测试我们发现：

- Widget的创建可以做到复用，通过const修饰
- Element并没有复用,其原因应该是在于isShow为false的时候导致其被deactivate 然后 unmount，从Element树种被移除掉。
- 有的人肯定有些疑问，怎么全程没看到activate呢？它不应该属于生命周期的一部分吗？这个就需要用到Key了，在接下来的课程里，讲到Key的时候，我们再详细的学习。

总结

本期我们对Widget，Element有了一个详细的认知，但其实它还有一个State类（StatefulWidget的核心实现）和RenderObject类，这两个下期我再分析。



2人点赞 >



📖 日记本



更多精彩内容，就在简书APP



"小礼物走一走，来简书关注我"

赞赏支持

还没有人赞赏，支持一下



i校长 抽象化是一种非常的不同于模糊化的东西 ... 抽象的目的并不是为了...

总资产332 共写了12.6W字 获得757个赞 共916个粉丝

关注

写下你的评论...

全部评论 1

只看作者

按时间倒序 按时间正序



Traxex_you

2楼 07.04 11:17

学习



被以下专题收入，发现更多相似内容



Android...



Android...



Flutter